

Graph Engineering for Everyone

*From seven bridges in Königsberg to multi-agent AI systems
that share what they know. A STELLA + Christian Macion
field guide.*

Christian Macion — AI Engineer · STELLA Office

2026

Shareable teaching workbook. Built in the ...from the Ground Up mentee-edition format. Primary sources: `_sources/Graph-Engineering-Athropic-Playbook.pdf`, `_sources/Graph-Engineering-Athropic-Karpathy-Loop.pdf`. Numbers tagged **V** verified, **B** background, **SCHEMATIC** schematic. Public-safe — STELLA brand tokens kept internal; this is the external teaching version.

WHO IS THIS FOR

Anyone who needs an LLM to reason across many documents, many agents, or both. You do not need a math background. If you have ever pasted a stack of papers into a chatbot and watched it quietly lose the connection between them, this book is for you.

It is also for the engineer who's tired of being told “just put it in the context” and the PM who's tired of vector-search demos that fall apart on the third question.

HOW TO READ IT

Seven chapters in three parts.

Part A • Why. Chapters 1–2 set up the 290-year-old idea and name the modern problem it solves.

Part B • How. Chapters 3–5 walk through anatomy, the four-stage pipeline, and the shared-memory pattern.

Part C • Where. Chapters 6–7 match the graph to the five agent patterns Anthropic ships and give you a ten-item production checklist.

Read top to bottom, or jump to Part C if you just want to know whether to build one.

EVIDENCE TAGS

Every figure in this book carries a chip in the corner.

V = **verified** against a primary source.

B = **background** context.

SCHEMATIC = **schematic** / **illustrative**.

Treat the spine as load-bearing; treat the numbers as anchors for the idea, not as gospel.

MAP

Part A • Why

1. The seven bridges of Königsberg
2. Why context windows are not enough

Part B • How

3. Anatomy of a knowledge graph
4. The four-stage pipeline
5. The shared-memory pattern

Part C • Where

6. Five agent patterns, five graph roles
7. Building & shipping a graph that holds up

Part A. WHY GRAPHS

Two chapters. One sets up the 290-year-old idea. The other names the modern problem the idea solves.

BOTTOM LINE A graph is two lists and a rule. Everything else is what you draw on top.

Chapter 1. The seven bridges of Königsberg

CHAPTER OBJECTIVES

After this chapter you will be able to:

- Name what a graph is, in one sentence.
- Recognize a graph when you see one — in code, in conversation, in your own notes.
- Spot the moment a list of facts stops being enough.

Prerequisites: None. This is the first chapter..

ORIENT: the big picture before the detail

In 1736 a mathematician named Leonhard Euler walked the city of Königsberg and asked a trivial-sounding question: can a tourist visit every one of the city's seven bridges exactly once and still return to the hotel? He solved it by refusing to think about the streets. He drew four dots — one for each land mass — and seven lines, one for each bridge. The dots are what he later called *vertices*. The lines are *edges*. The picture is a graph.

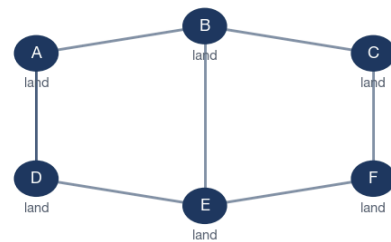
PRIME: new terms, one line each

- **Two lists and a rule:** A graph is two lists and a rule. List one is the *vertices*: the things. List two is the *edges*: the relationships. The rule says which things connect to which.

BRIDGE: from what you already own

Once you see a graph this way, every graph you've ever met fits the same picture: a friend network, a circuit board, a city street map, an org chart, a family tree, a map of your dependencies.

Königsberg, 1736: the bridges that started graph theory



Euler's question: walk every bridge exactly once and return home? Answer: no. The graph tells you why.

- B** The original graph. Four land masses, seven bridges, one famous non-tour. Euler's drawing is the prototype of every graph you will ever meet.

SHOW: worked example (skip if confident)

Vertices are the things. In conversation they might be people, projects, papers, companies, places, ideas, events. In code they are the rows of a database.

Edges are the relationships. In conversation they might be 'works on', 'depends on', 'was acquired by', 'contradicts'. In code they are the rows of another database — the ones with two foreign keys.

The rule is just the schema: what types of vertices are allowed, what types of edges, and what attributes each one carries.

FADE: same skill, you fill the blanks

Try it. Draw a graph of three projects you ran last month. Vertices are the projects. Pick two edges per project — one to a person who helped, one to a tool you couldn't do without.

If two of your projects have the same helper, the helper is in the graph twice — but they are the same person. That's the resolution problem we'll meet in chapter 4. If your edge “uses” points at different tools for different projects, you have already invented a *predicate vocabulary* — also chapter 4.

Answers: *The first lesson: drawing the graph forces you to decide what your schema allows. The moment you write down the rule, you can ship the pipeline.*

CHECK YOURSELF: cover the answers, recall from memory

From memory, before you read on:

1. Y
2. o
3. u
- 4.
5. h
6. a
7. v
8. e
- 9.
10. a
- 11.
12. l
13. i
14. s
15. t
- 16.
17. o
18. f
- 19.
20. 1
21. o
22. o
- 23.
24. c
25. o
26. m
27. p
28. a
29. n
30. i
31. e
32. s
- 33.
34. a
35. n
36. d
- 37.
38. a
- 39.
40. l
41. i
42. s
43. t
- 44.

45. o
46. f
47.
48. l
49. o
50. o
51. o
52.
53. a
54. c
55. q
56. u
57. i
58. s
59. i
60. t
61. i
62. o
63. n
64. s
65. .
66.
67. T
68. h
69. e
70.
71. a
72. c
73. q
74. u
75. i
76. s
77. i
78. t
79. i
80. o
81. n
82. s
83.
84. s
85. a
86. y
87.
88. '
89. A
90. c
91. m
92. e

93.
94. b
95. o
96. u
97. g
98. h
99. t
100.
101. B
102. e
103. t
104. a
105. '
106. .
107.
108. W
109. h
110. a
111. t
112.
113. d
114. o
115.
116. v
117. e
118. r
119. t
120. i
121. c
122. e
123. s
124.
125. a
126. n
127. d
128.
129. e
130. d
131. g
132. e
133. s
134.
135. l
136. o
137. o
138. k
139.
140. l

141. i
142. k
143. e
144. ?

Answers: Vertices are the 100 companies. Edges are the 1000 acquisitions. The rule says: a vertex is a company, an edge is named 'acquired', and every edge has a source and target (acquirer, acquired). That's it — that's a graph.

COMMON MISTAKE: you might think X; actually Y

People reach for a chart library the moment they hear the word 'graph'. They start drawing. The drawing is downstream. A graph is data; the visualization is the convenient artifact. Build the data first, draw later.

RECAP: back to altitude

- A graph is two lists and a rule: vertices, edges, schema.
- Euler's 1736 walk through Königsberg invented the picture; the data model is the same 290 years later.
- Draw the graph last. Build the data first.

BOTTOM LINE Context windows die. Files remember. The trick is remembering the right things.

Chapter 2. Why context windows are not enough

CHAPTER OBJECTIVES

After this chapter you will be able to:

- Explain why a model with a million-token context window still loses to a 200-line graph.
- Identify the failure mode where summarization gives you a confident wrong answer.
- State the rule of thumb for when to invest in a graph.

Prerequisites: Chapter 1 (graph as data)..

ORIENT: the big picture before the detail

Modern language models read a lot of text. The biggest can hold an entire novel, a thousand-page codebase, or the transcript of a long meeting. For one question that's enough. For fifty questions about the same material, it isn't.

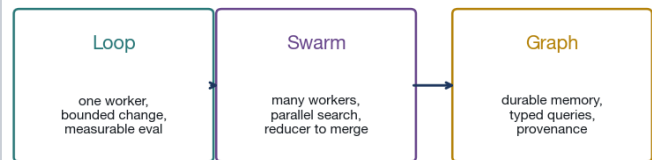
PRIME: new terms, one line each

- **Memory, not intelligence:** The bottleneck is memory. Ask an LLM five questions about a 200-page document. The first answer uses pages 1-200. The second answer uses your question + pages 1-200, minus whatever the model truncated to make room. By the fifth question, 'pages 1-200' is a memory of a memory of a memory. **Faithfulness drops with each hop.**

BRIDGE: from what you already own

Now multiply that by five worker agents each reading their own 200-page slice and handing a summary to the orchestrator. The orchestrator's context window fills with five summaries. The summaries did the chaining; the orchestrator only sees the result.

The three-stage progression: loop -> swarm -> graph



Each stage externalises a different bottleneck.

- B** Three steps. Step 1 — vibe coding: one model, one prompt, no state. Step 2 — agentic engineering: many models in a loop, context windows pass summaries. Step 3 — graph engineering: agents write to and read from a typed, queryable graph.

SHOW: worked example (skip if confident)

Vibe coding fails on second-use — the chat ends, the state is gone.

Agentic engineering fails on composition — five agents each summarize; their summaries don't link.

Graph engineering works — the graph is the durable layer that survives every context-window flush.

FADE: same skill, you fill the blanks

Try it. Open your last three conversations with any AI assistant. Now ask the assistant 'in which conversation did I first mention X?' If the assistant can answer, the platform is doing the graph work for you. If it can't, you have the problem this book is about.

Answers: Most assistants forget the moment the chat closes. Some offer a 'memory' toggle that does unstructured summarization, which drifts. The structured alternative is a graph of entities and relations — and that is what Anthropic's Knowledge Graph Cookbook ships out of the box.

CHECK YOURSELF: cover the answers, recall from memory

From memory, before you read on:

1. Y
2. o
3. u
- 4.
5. h
6. a
7. v
8. e
- 9.
10. o
11. n
12. e
- 13.
14. a
15. n
16. a
17. l
18. y
19. s
20. t
- 21.
22. a
23. g
24. e
25. n
26. t
- 27.
28. a
29. n
30. d
- 31.
32. o
33. n
34. e
- 35.
36. r
37. e
38. p
39. o
40. r
41. t
- 42.
43. a
44. g

45. e
46. n
47. t
48. .
49.
50. T
51. h
52. e
53.
54. a
55. n
56. a
57. l
58. y
59. s
60. t
61.
62. r
63. e
64. a
65. d
66. s
67.
68. 5
69. o
70.
71. f
72. i
73. l
74. i
75. n
76. g
77. s
78. ,
79.
80. t
81. h
82. e
83.
84. r
85. e
86. p
87. o
88. r
89. t
90.
91. w
92. r

93. i
94. t
95. e
96. s
97.
98. a
99.
100. s
101. u
102. m
103. m
104. a
105. r
106. y
107. .
108.
109. E
110. v
111. e
112. r
113. y
114.
115. m
116. o
117. r
118. n
119. i
120. n
121. g
122.
123. y
124. o
125. u
126.
127. w
128. a
129. n
130. t
131.
132. a
133.
134. f
135. r
136. e
137. s
138. h
139.
140. s

141. u
142. m
143. m
144. a
145. r
146. y
147.
148. t
149. h
150. a
151. t
152.
153. d
154. r
155. a
156. w
157. s
158.
159. o
160. n
161.
162. t
163. h
164. e
165.
166. l
167. a
168. s
169. t
170.
171. g
172. o
173.
174. d
175. a
176. y
177. s
178.
179. o
180. f
181.
182. f
183. i
184. l
185. i
186. n
187. g
188. s

189.
190. &
191. m
192. d
193. a
194. s
195. h
196. ;
197.
198. n
199. o
200. t
201.
202. j
203. u
204. s
205. t
206.
207. t
208. o
209. d
210. a
211. y
212. '
213. s
214. .
215.
216. W
217. h
218. e
219. r
220. e
221.
222. d
223. o
224. e
225. s
226.
227. t
228. h
229. e
230.
231. s
232. h
233. a
234. r
235. e
236. d

237.
238. s
239. t
240. a
241. t
242. e
243.
244. l
245. i
246. v
247. e
248. ?

Answers: *In a graph. The analyst writes the entities and relations from each filing into the graph. The report queries*

the graph each morning for what's been added in the last 90 days. Neither agent needs to remember the other agent's context window — the graph is the memory they both share.

COMMON MISTAKE: *you might think X; actually Y*

'Just dump everything into the context.' This works once. The 200K-context LLM giving you a confident wrong answer on question 47 is not a model bug — it's the inevitable failure of unbounded context.

RECAP: *back to altitude*

- The bottleneck is memory, not intelligence.
- Context windows die; files remember.
- A graph is the file format that lets agents share what they remember.

Part B. HOW

Three chapters. Anatomy, the four-stage pipeline, and the one pattern everyone builds.

BOTTOM LINE Every knowledge graph has three things: entities, relations, and provenance.

Chapter 3. Anatomy of a knowledge graph

CHAPTER OBJECTIVES

After this chapter you will be able to:

- Name the three core layers of a knowledge graph.
- Read a graph triple out loud.
- Spot the four places naive graphs fail.

Prerequisites: Chapters 1-2 (graph as data; why context windows are not enough)..

ORIENT: the big picture before the detail

You have heard the term 'knowledge graph' — usually attached to Google's search box, or a pharma company's drug-discovery platform. Here is the version that matters for AI agents: a small, typed, queryable store where every claim points back to the document it came from.

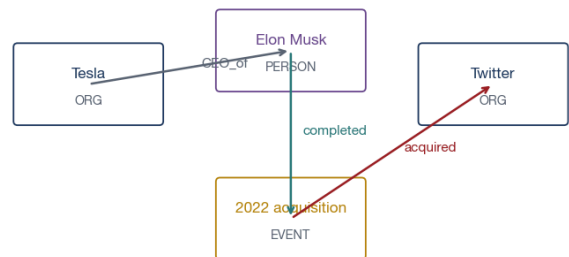
PRIME: new terms, one line each

- **Entities, relations, provenance:** Every knowledge graph is made of three things. **Entities** are the things — people, projects, papers, drugs, filings, claims. **Relations** are the typed edges between them. **Provenance** is the document and paragraph each claim came from. Skip provenance and you have a rumor graph.

BRIDGE: from what you already own

When you query this graph you get triples back: *source* — *predicate* — *target*, with a source-document ID attached. Every claim has a trace. Every trace is verifiable.

Anatomy of one knowledge graph (entities + relations + provenance)



Every edge carries a predicate (CEO_of, acquired, completed) and a source.

- **V** Anatomy. Three layers — entities, relations, provenance. Every edge carries a back-reference to the paragraph that produced it.

SHOW: worked example (skip if confident)

The entity. Has a stable id, a type (PERSON, PROJECT, DRUG), and a one-line description that disambiguates it from neighbors.

The relation. Has a source and target (both entity ids), a short verb-phrase predicate (works-on, contradicts, funded-by), and a back-reference to the source document.

The provenance. The document id, the paragraph offset, and the timestamp. Without this, claims are claims; with it, claims are evidence.

A triple, expressed as JSON:

```
{
  "source": "agent:alpha_researcher",
  "predicate": "owns",
  "target": "finding:graph-engineering-cookbook",
  "provenance": {
    "doc_id": "Graph-Engineering-Athropic-Playbook.",
    "offset": "lines 14-19",
    "verified_at": "2026-07-26"
  }
}
```

FADE: same skill, you fill the blanks

Try it. Pick three real entities from your work — a client, a project, a tool. Type one sentence describing each in plain English. Now pick two of the three and type one sentence about their relation. You have just authored a four-line knowledge graph.

Answers: Notice that one sentence per entity is enough. The Anthropic cookbook treats that one sentence as the disambiguation signal the resolver uses to merge duplicate names. Concise > clever.

CHECK YOURSELF: cover the answers, recall from memory

From memory, before you read on:

1. Y
2. o
3. u
- 4.
5. w
6. r
7. i
8. t
9. e
- 10.
11. '
12. A
13. l
14. i
15. c
16. e
- 17.
18. m
19. a
20. n
21. a
22. g
23. e
24. s
- 25.
26. B
27. e
28. t
29. a
30. '
31. .
- 32.
33. W
34. h
35. i
36. c
37. h
- 38.
39. s
40. i
41. d
42. e
- 43.
44. i

45. s
46.
47. t
48. h
49. e
50.
51. s
52. u
53. b
54. j
55. e
56. c
57. t
58. ,
59.
60. w
61. h
62. i
63. c
64. h
65.
66. s
67. i
68. d
69. e
70.
71. i
72. s
73.
74. t
75. h
76. e
77.
78. o
79. b
80. j
81. e
82. c
83. t
84. ,
85.

86. a
87. n
88. d
89.
90. w
91. h
92. e
93. r
94. e
95. '
96. s
97.
98. t
99. h
100. e
101.
102. p
103. r
104. o
105. v
106. e
107. n
108. a
109. n
110. c
111. e
112. ?

Answers: Alice is the subject (the source). Beta is the object (the target). 'Manages' is the predicate. And the provenance is the conversation, document, or filing where you read or wrote that sentence. If you can't point at the source, the claim isn't ready to enter the graph.

RECAP: back to altitude

- Entities, relations, provenance. The whole schema in one line.
- The hard part is resolution — turning 'Alice', 'A. Smith', 'Alice Smith', and 'Alice B. Smith' into one canonical node.
- That's the next chapter.

BOTTOM LINE Extract, resolve, assemble, query. Every knowledge graph in production is built by these four stages.

Chapter 4. The four-stage pipeline

CHAPTER OBJECTIVES

After this chapter you will be able to:

- Run the four stages by hand on a single document.
- Explain why resolution is where unsupervised graphs fail most often.
- Pick the right model per stage — Haiku for extraction, Sonnet for reasoning.

Prerequisites: Chapter 3 (anatomy of a knowledge graph)..

ORIENT: the big picture before the detail

Building a knowledge graph used to mean training a model. Building one today means writing four prompts. Each stage is an LLM call with a typed schema. The schema is the training data. The model itself is shared across every domain.

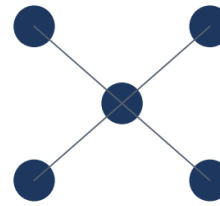
PRIME: new terms, one line each

- **One recipe, four stages:** **Stage 1.** Read a document. Extract entities (PERSON, ORG, LOC, EVENT, ARTIFACT) and the typed relations between them. Return as a typed object, not free text. **Stage 2.** Compare the new entities against the existing graph's alias map. Cluster any near-duplicates. **Stage 3.** Rewrite the relations in terms of the canonical entity ids. Pool hub-node mentions and summarize. **Stage 4.** At query time, take the user's question, extract its seed entity, walk 1–2 hops, hand the serialized triples to the answer model, and require every claim to cite a specific edge.

BRIDGE: from what you already own

The Anthropic cookbook collapses each stage to a single typed Claude API call. The Pydantic schema is the interface specification. Changing the schema is changing the system — not adding another moving part.

A graph = nodes + edges



Same structure whether 'node' means person, neuron, or page.

The same graph, three readings

knowledge graph: cities & roads
agent graph: agents & messages
computation graph: ops & data flow
neural net graph: neurons & weights

Different content; same mathematical object.

SCHEMATIC Three readings of the same graph. Top: nodes and edges. Middle: vertices and predicates. Bottom: typed triples. The data is the same; the framing determines what you can do with it.

SHOW: worked example (skip if confident)

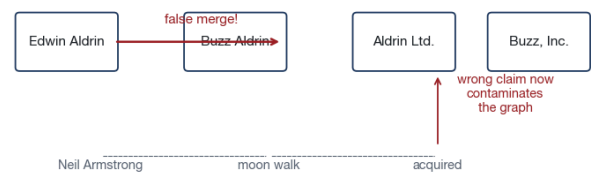
Extract with a cheap model. Haiku-class models are fast and accurate on extraction. Run them in batch on every document. Cache the prompt; cache the schema. The unit cost is fractions of a cent per document.

Resolve with a stronger model — only when needed. Block candidates by cheap signals (same last name, string overlap, embedding similarity). Send only ambiguous blocks (50-100 candidates) to a Sonnet-class model.

Assemble once. Every hub node (degree ≥ 3) gets a multi-paragraph profile pooled from every document that mentions it. Re-summarize *only* when the source-doc set changes.

Query per question. Walk $k=2$ hops from the question's seed entity, serialize the subgraph, hand it to the answer model, require a citation per claim. $k=1$ is fast but misses chains. $k=3+$ drowns the prompt.

Entity resolution: where catastrophic errors hide



Resolution must be additive, inspectable, reversible.

V Resolution, where unsupervised graphs most often fail. Left: four surface forms — 'Alice', 'A. Smith', 'Alice Smith', 'Alice B. Smith' — all pointing at the same person. Right: the canonical node after resolution, with all aliases mapped. Skip this stage and queries return one paragraph of duplicates.

Stage 1 — extract, in 8 lines:

```
from pydantic import BaseModel
from anthropic import Anthropic

class Entity(BaseModel):
    name: str
    type: str # PERSON | ORG | LOC | EVENT | ARTIF
    description: str # one-line, for resolution

def extract(client: Anthropic, text: str) -> list[Entity]:
    r = client.messages.parse(
        model="claude-haiku-4-5",
        max_tokens=2_048,
        messages=[{"role": "user",
                    "content": f"Extract entities from: {text}"},
                  ],
        output_format=list[Entity],
    )
    return r.parsed_output
```

FADE: same skill, you fill the blanks

Try it. Open one of your old notes — a meeting summary, a research memo, a project brief. Extract every person, every project, every decision. One sentence per item. Don't connect them yet.

Answers: If you got more than 30 entities you have probably over-extracted. The Anthropic cookbook treats the 'central only' instruction as precision-favoring. Trade recall for signal.

CHECK YOURSELF: cover the answers, recall from memory

From memory, before you read on:

1. Y
2. o
3. u
- 4.
5. h
6. a
7. v
8. e
- 9.
10. 1
11. O
12. ,
13. O
14. O
15. O
- 16.
17. d
18. o
19. c
20. u
21. m
22. e
23. n
24. t
25. s
- 26.
27. a
28. v
29. e
30. r
31. a
32. g
33. i
34. n
35. g
- 36.
37. 2
38. ,
39. O
40. O
41. O
- 42.
43. t
44. O

45. k
46. e
47. n
48. s
49.
50. e
51. a
52. c
53. h
54. .
55.
56. T
57. h
58. e
59.
60. e
61. x
62. t
63. r
64. a
65. c
66. t
67. i
68. o
69. n
70.
71. s
72. c
73. h
74. e
75. m
76. a
77.
78. i
79. s
80.
81. s
82. t
83. a
84. b
85. l
86. e
87. .
88.
89. T
90. h
91. e
92.

93. d
94. a
95. y
96. '
97. s
98.
99. r
100. e
101. s
102. o
103. l
104. u
105. t
106. i
107. o
108. n
109.
110. q
111. u
112. e
113. u
114. e
115.
116. i
117. s
118.
119. e
120. m
121. p
122. t
123. y
124. .
125.
126. W
127. h
128. a
129. t
130. '
131. s
132.
133. t
134. h
135. e
136.
137. d
138. o
139. m
140. i

- 141. n
- 142. a
- 143. n
- 144. t
- 145.
- 146. c
- 147. o
- 148. s
- 149. t
- 150. ?

Answers: Extraction. $10,000 \times 2,000$ tokens at Haiku rates (with prompt caching on the schema + instructions) is in the low single-digit dollars. Resolution is free when the queue is empty. Summarization only runs on the top-k hub nodes when

their source-doc set changes. The dominant cost always sits at the stage with the largest input.

COMMON MISTAKE: you might think X; actually Y

Trying to extract and resolve in one step. They are different cognitive loads. Extraction is literal reading; resolution is judgment. Split them. The schemas differ.

RECAP: back to altitude

- Extract, resolve, assemble, query. Each stage is a typed LLM call.
- The schema is the only training data you need.
- Run this on ten documents and you have a graph. Run it on ten thousand and you have infrastructure.

BOTTOM LINE The most useful graph in production is the one that five agents can read and write to at the same time.

Chapter 5. The shared-memory pattern

CHAPTER OBJECTIVES

After this chapter you will be able to:

- Explain why orchestrator-workers bottleneck on context.
- Draw the message-board pattern that replaces the bottleneck.
- List the three rules that keep the shared memory from drifting.

Prerequisites: Chapter 4 (the four-stage pipeline)..

ORIENT: the big picture before the detail

Picture one orchestrator delegating to five worker agents. Each worker reads its own 200-page slice and produces a summary. The orchestrator's job is to chain the summaries into a final answer. There's a problem: the orchestrator's context window fills with five summaries. The summaries did the chaining; the orchestrator only sees the result. When a sixth worker joins, the orchestrator falls over.

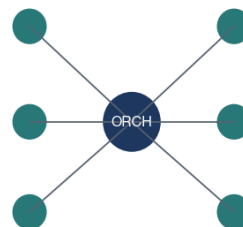
PRIME: new terms, one line each

- **Replace summaries with a graph:** Have each worker write entities and relations to a shared graph. The orchestrator's job is no longer to read summaries — it is to query the graph. The graph is the durable shared memory that survives every worker finishing. The orchestrator's context window stays small.

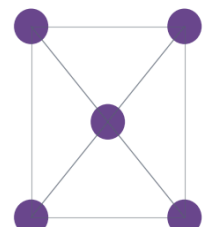
BRIDGE: from what you already own

Karpathy's AgentHub (2026) ships the same idea with even fewer moving parts: one git repo, one SQLite database, one Go binary. Agents push to the repo and post to the message board. The board is the shared memory; the repo is the shared artifact.

Hub-and-spoke (orchestrator)



Mesh (peer-to-peer)



B Two topologies. Left: hub-and-spoke. The orchestrator is every worker's only path to the rest of the world. It bottlenecks at the orchestrator's context window. Right: shared-memory mesh. Workers read and write the graph directly; the orchestrator only orchestrates queries.

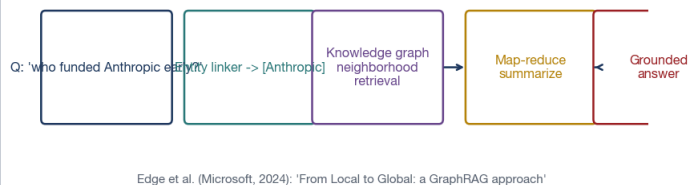
SHOW: worked example (skip if confident)

Rule 1: write before summarizing. Every worker writes typed triples to the graph first, then summarizes if needed. The graph is the authoritative state; summaries are derived.

Rule 2: read with provenance. Every worker reads with an edge-level provenance trail. If the answer would have to invent an edge, refuse.

Rule 3: schema-enforce at write time. If the schema doesn't allow an entity type or a predicate, reject the write at the door. Schema drift is worse than no graph.

GraphRAG: graph-augmented retrieval (Edge et al., 2024)



V GraphRAG retrieval. Top: a question seeds a subgraph ($k=2$ hops from the seed entity). Bottom: the grounded answer cites specific edges from specific documents. RAG retrieves passages; GraphRAG retrieves — and chains — facts.

Serialized subgraph, fed to the answer model:

```
def ask(question, graph_context):
    prompt = f"""
    Answer using ONLY the knowledge graph below.
    Cite the specific edges that support your answer.

    {graph_context}

    Question: {question}"""
    return call_model(prompt)
```

FADE: same skill, you fill the blanks

Try it. Build a graph of three real projects and the people who work on each. Pick one project; ask 'who on this project also worked on Project X?'

Answers: If you walked the graph by hand, congratulations — you've reinvented the $k=2$ -hop subgraph. The graph lets the LLM do the walking for you, with provenance, at scale.

CHECK YOURSELF: cover the answers, recall from memory

From memory, before you read on:

1. A
2. n
- 3.
4. e
5. v
6. a
7. l
8. u
9. a
10. t
11. o
12. r
- 13.
14. a
15. g
16. e
17. n
18. t
- 19.
20. r
21. e
22. c
23. e
24. i
25. v
26. e
27. s
- 28.
29. a
- 30.
31. g
32. e
33. n
34. e
35. r
36. a
37. t
38. o
39. r
40. '
41. s
- 42.
43. c
44. l

45. a
46. i
47. m
48. .
49.
50. I
51. t
52.
53. m
54. u
55. s
56. t
57.
58. c
59. h
60. e
61. c
62. k
63.
64. t
65. h
66. e
67.
68. c
69. l
70. a
71. i
72. m
73.
74. a
75. g
76. a
77. i
78. n
79. s
80. t
81.
82. e
83. x
84. t
85. r
86. a
87. c
88. t
89. e
90. d
91.
92. f

93. a
94. c
95. t
96. s
97. ,
98.
99. n
100. o
101. t
102.
103. e
104. s
105. t
106. i
107. m
108. a
109. t
110. e
111. d
112.
113. o
114. n
115. e
116. s
117. .
118.
119. W
120. h
121. a
122. t
123. '
124. s
125.
126. t
127. h
128. e
129.
130. c
131. h
132. e
133. a
134. p
135. e
136. s
137. t
138.
139. w
140. a

141. y
142.
143. t
144. o
145.
146. w
147. i
148. r
149. e
150.
151. t
152. h
153. a
154. t
155. ?

Answers: Query the graph for the specific triple the generator claimed, with the predicate and the source

document. If the edge exists with provenance, the claim is grounded. If not, the evaluator flags it. The graph is the grounding layer for evaluator-optimizer loops.

COMMON MISTAKE: you might think X; actually Y

Letting two workers write the same entity under different aliases. Resolution (chapter 4) is the gate. Without it, the orchestrator's query returns duplicate answers and nobody knows which one to trust.

RECAP: back to altitude

- Replace summaries with a graph.
- Workers write typed triples, the orchestrator queries the graph, the answer cites the edges.
- The graph is the shared memory. The schema is the trust boundary.

Part C. WHERE

Two chapters. Five agent patterns with a graph in each, and a production-readiness checklist.

BOTTOM LINE The graph has five jobs to do, one for every agent pattern Anthropic ships.

Chapter 6. Five agent patterns, five graph roles

CHAPTER OBJECTIVES

After this chapter you will be able to:

- Name the five agent patterns Anthropic publishes.
- Match each pattern to the role a graph plays in it.
- Recognize which pattern is which in the wild.

Prerequisites: Chapter 5 (shared-memory pattern)..

ORIENT: *the big picture before the detail*

Anthropic's *Building Effective Agents* essay (Dec 2024) lists five canonical patterns. They are the vocabulary for everything that follows. Re-read them as *control flow* shapes first; you can swap a graph into each one.

PRIME: *new terms, one line each*

- **Five patterns, one shared backbone:** The patterns are: augmented LLM, prompt chaining, routing, orchestrator-workers, evaluator-optimizer. Below them the graph plays a different role each time — but it is *the same graph*.

BRIDGE: *from what you already own*

The power of the five-pattern vocabulary is that you don't have to choose. A system that starts as a single augmented LLM can grow into orchestrator-workers without a rewrite — the graph layers in beneath it.

Pick the topology for the lifetime of the subgraph

Chain	stable steps; one artifact per step
Router	different policies for different inputs
Parallel	independent subtasks; voting or sectioning
Orch-workers	dynamic decomposition; subtasks vary
Eval-optim	two-role loop; one generates, one judges

B Five graph topologies that pair with the five agent patterns: star (augmented LLM), DAG (chaining), tree (routing), mesh (orchestrator-workers), and feedback loop (evaluator-optimizer).

SHOW: worked example (skip if confident)

Augmented LLM. Graph role: retrieval source. The LLM queries the graph in addition to whatever vector store it already has. Use this when you have lots of facts and need to ground every answer.

Prompt chaining. Graph role: gate signal. Between two chain steps, a graph query decides whether to branch. Use this when the chain is fragile and you want data-driven handoffs.

Routing. Graph role: classifier input. The router consults the graph to learn what kinds of inputs each specialist handles. Use this when specialists need to be added or removed without retraining a router model.

Orchestrator-workers. Graph role: shared memory. This is the chapter-5 pattern. Workers read and write the graph; the orchestrator queries it.

Evaluator-optimizer. Graph role: grounding layer. The evaluator checks the generator's claims against the graph's edges with provenance, not against model estimation.

FADE: same skill, you fill the blanks

Try it. Take your current system — or the system you most want to build — and pick the one pattern that fits it best. Then identify what you'd want from a graph at that level.

Answers: *Augmented LLM users want a fast retrieval source with good recall. Orchestrator-workers users want a typed shared memory that survives context flushes. Evaluator-optimizer users want a grounding layer with edge-level provenance. Different roles; same graph.*

CHECK YOURSELF: cover the answers, recall from memory

From memory, before you read on:

1. Y
2. o
3. u
4. '
5. r
6. e
- 7.
8. b
9. u
10. i
11. l
12. d
13. i
14. n
15. g
- 16.
17. '
18. s
19. e
20. a
21. r
22. c
23. h
- 24.
25. o
26. v
27. e
28. r
- 29.
30. c
31. o
32. m
33. p
34. a
35. n
36. y
- 37.
38. f
39. i
40. l
41. i
42. n
43. g
44. s

45. '
46. .
47.
48. M
49. o
50. s
51. t
52.
53. f
54. i
55. l
56. i
57. n
58. g
59. s
60.
61. a
62. r
63. e
64.
65. l
66. o
67. n
68. g
69.
70. P
71. D
72. F
73. s
74. .
75.
76. T
77. h
78. e
79.
80. u
81. s
82. e
83. r
84.
85. a
86. s
87. k
88. s
89.
90. '
91. w
92. h

93. a
94. t
95.
96. d
97. i
98. d
99.
100. c
101. o
102. m
103. p
104. a
105. n
106. y
107.
108. X
109.
110. o
111. w
112. n
113.
114. i
115. n
116.
117. Q
118. 3
119.
120. 2
121. o
122. 2
123. 4
124. ?
125. '
126.
127. S
128. i
129. n
130. g
131. l
132. e
133. -
134. h
135. o
136. p
137.
138. a
139. n
140. s

141. w
142. e
143. r
144. ,
145.
146. m
147. u
148. l
149. t
150. i
151. -
152. h
153. o
154. p
155.
156. q
157. u
158. e
159. s
160. t
161. i
162. o
163. n
164. .
165.
166. W
167. h
168. i

169. c
170. h
171.
172. p
173. a
174. t
175. t
176. e
177. r
178. n
179. ?

Answers: *Augmented LLM with the graph as a retrieval source. Augmented LLM covers the single-hop part; the graph covers the multi-hop part (X owned Y; Y owned Z). The graph hands the LLM a tiny subgraph instead of a long context.*

COMMON MISTAKE: *you might think X; actually Y*

Building one of these patterns without the graph and then trying to bolt the graph on later. The graph is cheaper to design in from day one — post-hoc retrofitting is what gets you schema-drift and duplicate-entity bugs that nobody noticed until production.

RECAP: *back to altitude*

- Five patterns, one graph. Each pattern consumes the graph differently.
- Each role is an upgrade to the pattern you already have — not a replacement.

BOTTOM LINE A production graph is the one you can debug at 3 a.m. when a wrong answer pages you.

Chapter 7. Building & shipping a graph that holds up

CHAPTER OBJECTIVES

After this chapter you will be able to:

- List the ten production-readiness checks.
- Identify which failure each check prevents.
- Recognize which one you're missing when something breaks.

Prerequisites: Chapter 6 (five patterns, five roles)..

ORIENT: *the big picture before the detail*

Almost every knowledge-graph project dies in production the same way: a wrong answer surfaces, nobody can trace it, and the team retreats to vector search. The retreat is the symptom. The cause is shipping without a production checklist.

PRIME: *new terms, one line each*

- **Ten checks, before each ship:** Gold set, alias map, schema version, extraction cap, resolution fallback, provenance tracking, incremental update, connectivity monitor, summarization trigger, human sample. Ten items. Run all ten. Don't ship without them.

BRIDGE: *from what you already own*

The Anthropic cookbook calls the first three 'evaluation'. The next three are 'silent corruption guards'. The last four are 'operational discipline'. Skip any of the three groups and you'll know why your graph broke — just after the post-mortem.

Production checklist (10 elements)

- ☐ Objective defined; success verifiable
- ☐ Distinguishable metric
- ☐ Updates reversible
- ☐ Tool schemas typed
- ☐ Artifact contract pinned
- ☐ Every claim has a source
- ☐ Resolution policy reversible
- ☐ Budgets explicit
- ☐ Metrics tracked over time
- ☐ Can resume from any state

V The ten checks at a glance. Three for evaluation, three for silent-corruption guards, four for operational discipline. Run all ten before each ship.

SHOW: worked example (skip if confident)

1. Gold set. A hand-labeled corpus of 20-50 documents — what entities should be there, what relations should link them. Score F1 against the gold set every build.

2. Alias map. Every canonical form a resolver might produce, matched to its surface variants. The cookbook shows the failure mode: 'Neil Alden Armstrong' resolves but the scorer doesn't recognize the gold form.

3. Schema version. A versioned file under version control, alongside the graph. Entities extracted under schema v1 must be re-extractable under schema v2.

4. Extraction cap. A per-run limit on documents processed. Unbounded cost is the silent failure of unsupervised pipelines.

5. Resolution fallback. Every raw name should land in exactly one cluster — including single-element clusters when no match exists. Silent entity loss is worse than false merges.

6. Provenance tracking. Every edge has a source document id, an extraction timestamp, and the entity-id pair that produced it.

7. Incremental update. New documents land at the edge of the graph; only changed hub nodes get re-summarized. Rebuilding the whole graph per doc is a cost cliff.

8. Connectivity monitor. Connected-component count per session. A growing number of components means resolution is failing somewhere.

9. Summarization trigger. Re-summarize only when the source-doc set for that entity changes. Ad-hoc triggers are wasted Sonnet calls.

10. Human sample. Someone reads one random node profile per day. Comprehension rot is the failure mode you can't catch programmatically.

FADE: same skill, you fill the blanks

Try it. Pick your current project. Score it 0-10 on the checklist. The three items you scored zero on are the three failure modes that will page you at 3 a.m.

Answers: *If you scored below 6, you're not alone — the cookbook openly admits 'a pipeline missing any of them has a specific, nameable risk that will surface eventually.'*

CHECK YOURSELF: cover the answers, recall from memory

From memory, before you read on:

1. Y
2. o
3. u
4. r
- 5.
6. g
7. r
8. a
9. p
10. h
- 11.
12. s
13. h
14. i
15. p
16. s
17. .
- 18.
19. S
20. i
21. x
- 22.
23. w
24. e
25. e
26. k
27. s
- 28.
29. l
30. a
31. t
32. e
33. r
- 34.
35. t
36. h
37. e
- 38.
39. a
40. n
41. s
42. w
43. e
44. r

45. s
46.
47. a
48. r
49. e
50.
51. g
52. e
53. t
54. t
55. i
56. n
57. g
58.
59. w
60. o
61. r
62. s
63. e
64. .
65.
66. T
67. h
68. e
69.
70. c
71. a
72. u
73. s
74. e
75.
76. i
77. s
78.
79. t
80. h
81. e
82.
83. s
84. o
85. u
86. r
87. c
88. e
89.
90. d
91. o
92. c

93. u
94. m
95. e
96. n
97. t
98. s
99.
100. s
101. h
102. i
103. f
104. t
105. i
106. n
107. g
108.
109. u
110. n
111. d
112. e
113. r
114. n
115. e
116. a
117. t
118. h
119.
120. t
121. h
122. e
123.
124. c
125. a
126. c
127. h
128. e
129. d
130.
131. h
132. u
133. b
134. -
135. n
136. o
137. d
138. e
139.
140. s

141. u
142. m
143. m
144. a
145. r
146. i
147. e
148. s
149. .
150.
151. W
152. h
153. i
154. c
155. h
156.
157. c
158. h
159. e
160. c
161. k
162.
163. c
164. a
165. t
166. c
167. h
168. e
169. s
170.
171. i
172. t
173. ?

Answers: Check #9, the summarization trigger. Re-summarize should fire whenever the source-doc set for a hub node changes. If you triggered re-summarization ad-hoc, the schedule drifted. Fix: hook the summarizer to the document-ingestion event, not to the calendar.

COMMON MISTAKE: you might think X; actually Y

Treating the ten checks as 'we'll do them after we ship the first version'. Every one of them is cheaper to add at build time than to retrofit.

RECAP: back to altitude

- Ten checks. Run all ten before each ship.
- Three groups: evaluation, silent-corruption guards, operational discipline.
- The evaluation harness, provenance tracking, and human sample are the judgment; everything else is infrastructure.

THE RECAP IN ONE PAGE

RECAP: back to altitude

- A graph is two lists and a rule — vertices, edges, schema. That's all.
- Memory is the bottleneck. Context windows die; files remember. A graph is the file format that lets agents share what they remember.
- Every knowledge graph in production is built by four stages: extract, resolve, assemble, query. Each is a typed LLM call. The schema is the training data.
- The most useful graph in production is the one five agents can read and write at the same time without the orchestrator bottleneck.
- The graph plays five different roles in Anthropic's five agent patterns — but it is the same graph.
- Ten checks before each ship. The evaluation harness, provenance tracking, and human sample are the judgment; everything else is infrastructure.

VERDICT

ORIENT: the big picture before the detail

Graph engineering is the layer beneath agentic engineering. If you ship agents, you owe them a graph. If you don't, you'll keep relitigating what one agent 'meant' in the context window above another agent's head.

THE PROGRESSION

ORIENT: *the big picture before the detail*

Vibe coding — one prompt, one answer, no state. Demo quality.

Agentic engineering — orchestrated multi-step workflows with tools and skills. The orchestrator's context window is the fragile link.

Graph engineering — agents share a typed, queryable graph as their persistent world model. The graph is the layer that survives every context flush.

GLOSSARY

SECTION GLOSSARY

graph: Two lists and a rule: vertices, edges, schema. **vertex / node:** One of the things in a graph. **edge:** A typed relationship between two vertices. **schema:** The rule — which entity types, which predicates. **alias map:** The dictionary of surface forms — canonical ids. **provenance:** The source document a fact was extracted from.

subgraph: A portion of the graph, taken around a seed entity. **hub node:** An entity that appears in many documents. **resolution:** The step that merges duplicates into one canonical id.

GraphRAG: Retrieval-augmented generation over a knowledge graph. **orchestrator-workers:** One agent delegates to many, the classic Anthropic pattern. **evaluator-optimizer:** Two agents in a loop — one generates, one grades. **grounding:** Checking that a claim is supported by an extracted fact, not by model estimation. **triple:** A (source, predicate, target) tuple with provenance.

WHAT TO READ NEXT

ORIENT: *the big picture before the detail*

Primary sources. Anthropic's *Knowledge Graph Cookbook* (independent synthesis, Graph-Engineering-Athropic-Playbook.pdf, July 2026) and Anthropic's *Building Effective Agents* essay (Erik S. & Barry Zhang, Dec 2024). Microsoft's *From Local to Global: A Graph RAG Approach* (Edge et al., arXiv:2404.16130, April 2024).

Companion volumes. *AI Engineering from the Ground Up* — the workbook this one grew out of.